

XArmAPI 完整函式與屬性參考文件

基於 `xarm_api.py` 原始碼逐行對照整理，涵蓋所有 `@property`、method、以及 alias 別名。

傳回值中 `code == 0` 表示執行成功；非零值請參閱官方 API Code 文件。

角度單位：預設為度 ($^{\circ}$)；若建構時傳入 `is_radian=True`，則全面使用弧度 (rad)。

目錄

1. 建構子 `init`
2. 連線管理
3. 狀態與模式屬性
4. 位置與角度屬性
5. 速度、加速度、限制屬性
6. 感測器與診斷屬性
7. 安全模式屬性
8. 運動控制方法
9. 速度控制方法 (Velocity Control)
10. 參數設定方法
11. 狀態查詢方法
12. 運動學工具方法
13. 末端夾爪 — xArm Gripper
14. 末端夾爪 — Vacuum Gripper
15. 末端夾爪 — Robotiq Gripper
16. 末端夾爪 — BIO Gripper
17. 末端夾爪 — 其他夾爪
18. Tool GPIO (tgpio)
19. Controller GPIO (cgpio)
20. 六軸力矩感測器 (FT Sensor)
21. Linear Motor (線性馬達)
22. Callback 回呼管理
23. 軌跡錄製與回放
24. 校正工具方法
25. RS485 / Modbus RTU
26. Modbus TCP (標準 Modbus)
27. 縮減模式與安全邊界
28. 進階診斷與錯誤資訊
29. DH 參數、回饋、其他進階功能
30. Lite6 專屬夾爪
31. 偵錯與版本查詢

1. 建構子

```
__init__(port=None, is_radian=False, do_not_open=False, **kwargs)
```

建立 `XArmAPI` 實例並 (預設) 自動連線。

參數	型別	預設	說明
<code>port</code>	str	None	機械臂 IP 位址, 如 <code>'192.168.1.185'</code> ; <code>do_not_open=False</code> 時必須填寫
<code>is_radian</code>	bool	False	全域角度單位。 <code>False</code> =度 (°) , <code>True</code> =弧度 (rad) 。影響所有帶角度參數的 method 與 property
<code>do_not_open</code>	bool	False	<code>True</code> 時不自動連線, 需手動呼叫 <code>connect()</code>
<code>axis</code>	int (kwargs)	7	軸數, 序列埠連線時才需要指定
<code>check_tcp_limit</code>	bool (kwargs)	False	是否檢查 <code>set_position</code> / <code>move_arc_lines</code> 中 roll/pitch/yaw 超限
<code>check_joint_limit</code>	bool (kwargs)	True	是否檢查 <code>set_servo_angle</code> / <code>set_servo_angle_j</code> 超限
<code>check_cmdnum_limit</code>	bool (kwargs)	True	是否檢查指令佇列數量超限
<code>max_cmdnum</code>	int (kwargs)	512	最大佇列指令數, 僅在 <code>check_cmdnum_limit=True</code> 時有效
<code>check_is_ready</code>	bool (kwargs)	True	是否在動作前確認機械臂就緒, 僅在 firmware < 1.5.20 時有效

角度單位換算： $1\text{ rad} = 57.296^\circ$; $1^\circ = 0.01745\text{ rad}$

2. 連線管理

```
connect(port=None, baudrate=None, timeout=None, axis=None, **kwargs)
```

連線至 xArm 控制器。

參數	說明
<code>port</code>	IP 位址，預設使用建構時的值
<code>baudrate</code>	串列埠鮑率（序列埠連線使用，通常不需要設定）
<code>timeout</code>	串列埠逾時（通常不需要設定）
<code>axis</code>	軸數，序列埠連線時才需要

`disconnect()`

斷開與 xArm 的連線。無參數，無傳回值。

`send_cmd_sync(command=None)`

發送文字指令並等待回應（不等待動作完成）。

參數	說明
<code>command</code>	G-code 或 H-code 字串，例如 <code>'G1 X300 Y0 Z200 F100'</code>

支援的指令格式（節錄）：

指令	對應 API
<code>G1 X{x} Y{y} Z{z} A{roll} B{pitch} C{yaw} F{speed} Q{acc}</code>	<code>set_position</code> (MoveLine)
<code>G7 I{J1} J{J2} ... F{speed}</code>	<code>set_servo_angle</code>
<code>G8 F{speed}</code>	<code>move_gohome</code>
<code>H11 I{servo_id} V{enable}</code>	<code>motion_enable</code>
<code>H12 V{state}</code>	<code>set_state</code>
<code>H19 V{mode}</code>	<code>set_mode</code>

3. 狀態與模式屬性

屬性名稱	型別	說明
<code>connected</code>	bool	是否已連線
<code>default_is_radian</code>	bool	建構時設定的角度單位 (<code>is_radian</code> 值)
<code>state</code>	int	機械臂狀態: 1=運動中、2=睡眠、3=暫停、4=停止中
<code>mode</code>	int	控制模式 (見下表) , 需 socket 連線且 <code>enable_report=True</code>
<code>is_simulation_robot</code>	bool	是否為模擬機器人
<code>has_err_warn</code>	bool	是否有錯誤或警告
<code>has_error</code>	bool	是否有錯誤
<code>has_warn</code>	bool	是否有警告
<code>error_code</code>	int	控制器錯誤碼
<code>warn_code</code>	int	控制器警告碼
<code>cmd_num</code>	int	控制器指令快取數量
<code>device_type</code>	int	裝置類型 (需 rich report)
<code>axis</code>	int	軸數 (需 rich report)
<code>master_id</code>	int	Master ID (需 rich report)
<code>slave_id</code>	int	Slave ID (需 rich report)
<code>is_reduced_mode</code>	bool	是否處於縮減模式
<code>is_fence_mode</code>	bool	是否處於安全邊界模式
<code>is_report_current</code>	bool	是否回報電流
<code>is_approx_motion</code>	bool	是否允許近似運動
<code>is_cart_continuous</code>	bool	笛卡爾運動是否連續
<code>is_collision_rebound</code>	bool	碰撞反彈是否啟用

mode 值說明:

值	模式
0	位置控制模式（預設）
1	伺服運動模式（需搭配 <code>set_servo_angle_j</code> / <code>set_servo_cartesian</code> ）
2	關節示教模式
3	笛卡爾示教模式（無效）
4	關節速度控制模式
5	笛卡爾速度控制模式
6	關節線上軌跡規劃模式
7	笛卡爾線上軌跡規劃模式

4. 位置與角度屬性

屬性名稱	型別	說明
<code>position</code>	list	笛卡爾位置 <code>[x(mm), y(mm), z(mm), roll, pitch, yaw]</code> ，角度依 <code>default_is_radian</code>
<code>position_aa</code>	list	軸角表示位姿 <code>[x(mm), y(mm), z(mm), rx, ry, rz]</code>
<code>angles</code>	list	各關節角度 <code>[J1, J2, ..., J7]</code> ，依 <code>default_is_radian</code>
<code>last_used_position</code>	list	上次 <code>set_position</code> 使用的位姿，作為下次的預設值
<code>last_used_angles</code>	list	上次 <code>set_servo_angle</code> 使用的角度，作為下次的預設值
<code>tcp_offset</code>	list	TCP 偏移 <code>[x, y, z, roll, pitch, yaw]</code> （需 socket + enable_report）
<code>world_offset</code>	list	世界座標偏移 <code>[x, y, z, roll, pitch, yaw]</code> （需 firmware > 1.2.11）
<code>gravity_direction</code>	list	重力方向（需 rich report）
<code>ft_ext_force</code>	list	六軸力矩感測器外力偵測值（補償後）
<code>ft_raw_force</code>	list	六軸力矩感測器原始讀值（未補償）

5.速度、加速度、限制屬性

屬性名稱	型別	說明
<code>tcp_jerk</code>	float	TCP jerk (mm/s^3)
<code>tcp_speed_limit</code>	list	TCP 速度限制 <code>[min(mm/s), max(mm/s)]</code> (需 rich report)
<code>tcp_acc_limit</code>	list	TCP 加速度限制 <code>[min(mm/s²), max(mm/s²)]</code> (需 rich report)
<code>last_used_tcp_speed</code>	float	上次 <code>set_position</code> 使用的速度 (mm/s)
<code>last_used_tcp_acc</code>	float	上次 <code>set_position</code> 使用的加速度 (mm/s^2)
<code>joint_jerk</code>	float	關節 jerk ($^\circ/\text{s}^3$ 或 rad/s^3)
<code>joint_speed_limit</code>	list	關節速度限制 (需 rich report)
<code>joint_acc_limit</code>	list	關節加速度限制 (需 rich report)
<code>last_used_joint_speed</code>	float	上次 <code>set_servo_angle</code> 使用的速度
<code>last_used_joint_acc</code>	float	上次 <code>set_servo_angle</code> 使用的加速度
<code>reduced_max_tcp_speed</code>	float	縮減模式下的最大 TCP 速度 (mm/s)
<code>reduced_max_joint_speed</code>	float	縮減模式下的最大關節速度
<code>reduced_tcp_boundary</code>	list	縮減模式的 TCP 邊界 <code>[x_max, x_min, y_max, y_min, z_max, z_min]</code>
<code>reduced_joint_limits</code>	list	縮減模式的關節限位 <code>[[J1_min, J1_max], ..., [J7_min, J7_max]]</code>

6. 感測器與診斷屬性

屬性名稱	型別	說明
version	str	xArm 韌體版本
version_number	tuple	(major, minor, revision)
sn	str	xArm 序號
control_box_sn	str	控制箱序號
temperatures	list	各伺服馬達溫度 [T1, ..., T7] (需 firmware > 1.2.11)
voltages	list	各伺服馬達電壓 [V1, ..., V7]
currents	list	各伺服馬達電流 [I1, ..., I7]
joints_torque	list	各關節力矩 (需 rich report)
tcp_load	list	TCP 負載 [weight(kg), [x(mm), y(mm), z(mm)]] (需 rich report)
collision_sensitivity	int	碰撞靈敏度 0~5 (需 rich report)
teach_sensitivity	int	示教靈敏度 1~5 (需 rich report)
motor_brake_states	list	馬達剎車狀態 (0=啟用, 1=停用) , 需 rich report
motor_enable_states	list	馬達啟用狀態 (0=停用, 1=啟用) , 需 rich report
servo_codes	list	伺服狀態與錯誤碼 [[status, code], ...] (含 tool GPIO)
realtime_tcp_speed	float	TCP 即時速度 (mm/s) , 需 firmware > 1.2.11
realtime_joint_speeds	list	各關節即時速度 (需 firmware > 1.2.11)
count	int	計數器數值
cgpio_states	list	控制器 GPIO 狀態 (詳見 get_cgpio_state)
self_collision_params	list	自碰撞參數 [detection_on, tool_type, model_params]
gpio_reset_config	list	GPIO 重置設定 [cgpio_reset_enable, tgpio_reset_enable]

7. 安全模式屬性

屬性名稱	說明
<code>only_check_result</code>	僅檢查結果（內部用）
<code>report_data</code>	原始 report 資料
<code>arm</code>	XArm 內部實作實例（不建議直接使用）
<code>core</code>	核心層 API，進階開發者用（如 <code>self.core.move_line()</code> ）

8. 運動控制方法

`get_position(is_radian=None) → (code, [x, y, z, roll, pitch, yaw])`

取得目前笛卡爾位姿。

參數	說明
<code>is_radian</code>	回傳 roll/pitch/yaw 是否用弧度，預設 <code>default_is_radian</code>

`set_position(x, y, z, roll, pitch, yaw, radius=None, speed=None, mvacc=None, mvtime=None, relative=False, is_radian=None, wait=False, timeout=None, **kwargs) → code`

設定笛卡爾目標位姿（基座座標系）。未填參數使用 `last_used_position` 的對應值。

參數	說明
<code>x/y/z</code>	目標位置 (mm)
<code>roll/pitch/yaw</code>	姿態角 (°或rad) , 分別繞 X/Y/Z 軸旋轉
<code>radius</code>	軌跡半徑: <code>None</code> 或 <code>< 0</code> → MoveLine (直線) ; <code>>= 0</code> → MoveArcLine (弧線插補)
<code>speed</code>	移動速度 (mm/s) , 預設 <code>last_used_tcp_speed</code>
<code>mvacc</code>	加速度 (mm/s ²) , 預設 <code>last_used_tcp_acc</code>
<code>mvtime</code>	保留, 填 0
<code>relative</code>	是否為相對移動
<code>is_radian</code>	roll/pitch/yaw 是否為弧度
<code>wait</code>	是否等待動作完成後才返回
<code>timeout</code>	最大等待秒數 (僅 wait=True 時有效)

```
set_tool_position(x=0, y=0, z=0, roll=0, pitch=0, yaw=0, speed=None, mvacc=None, mvtime=None, is_radian=None, wait=False, timeout=None, radius=None, **kwargs) → code
```

相對**工具座標系**移動 (座標系隨末端位姿變化) 。

參數	說明
<code>x/y/z</code>	相對工具座標系的位移量 (mm) , 預設 0
<code>roll/pitch/yaw</code>	相對工具座標系的旋轉量, 預設 0
<code>radius</code>	同 <code>set_position</code> , 需 firmware <code>>= 1.11.100</code>
<code>speed/mvacc/wait/timeout</code>	同 <code>set_position</code>

```
get_servo_angle(servo_id=None, is_radian=None, is_real=False) → (code, angles 或 angle)
```

取得關節角度。

參數	說明
<code>servo_id</code>	1~軸數：取單一關節角度；None 或 8：取全部關節角度
<code>is_radian</code>	回傳是否用弧度
<code>is_real</code>	是否取即時值（非緩衝值）

`set_servo_angle(servo_id=None, angle=None, speed=None, mvacc=None, mvtime=None, relative=False, is_radian=None, wait=False, timeout=None, radius=None, **kwargs) → code`

設定關節角度目標。

參數	說明
<code>servo_id</code>	1~軸數：控制單一關節， <code>angle</code> 為數值；None/8：全部關節， <code>angle</code> 為 list
<code>angle</code>	目標角度（°或rad）或角度 list
<code>speed</code>	關節速度（°/s 或 rad/s）
<code>mvacc</code>	關節加速度（°/s ² 或 rad/s ² ）
<code>relative</code>	相對移動
<code>radius</code>	混合半徑（需 firmware > 1.5.20），不可大於軌跡長度
<code>wait/timeout</code>	同 <code>set_position</code>

`set_servo_angle_j(angles, speed=None, mvacc=None, mvtime=None, is_radian=None, **kwargs) → code`

伺服模式下設定關節角度（只執行最後一筆指令）。

⚠ 必須先呼叫 `set_mode(1)` 切換至伺服運動模式。

不修改 `last_used_angles` 等快取值。

參數	說明
<code>angles</code>	角度 list <code>[J1, J2, ..., J7]</code> （°或rad）

`set_servo_cartesian(mvpose, speed=None, mvacc=None, mvtime=0, is_radian=None, is_tool_coord=False, **kwargs) → code`

伺服模式下設定笛卡爾位姿（只執行最後一筆指令）。

⚠ 必須先呼叫 `set_mode(1)`。

參數	說明
<code>mvpose</code>	目標位姿 <code>[x, y, z, roll, pitch, yaw]</code>
<code>is_tool_coord</code>	是否使用工具座標系

```
set_servo_cartesian_aa(axis_angle_pose, speed=None, mvacc=None, is_radian=None, is_tool_coord=False, relative=False, **kwargs) → code
```

伺服模式下，以**軸角（axis-angle）**表示法設定笛卡爾位姿。
需 firmware >= 1.4.7，且先呼叫 `set_mode(1)`。

參數	說明
<code>axis_angle_pose</code>	<code>[x(mm), y(mm), z(mm), rx, ry, rz]</code>
<code>is_tool_coord</code>	是否使用工具座標系
<code>relative</code>	相對移動

```
move_circle(pose1, pose2, percent, speed=None, mvacc=None, mvtime=None, is_radian=None, wait=False, timeout=None, is_tool_coord=False, is_axis_angle=False, **kwargs) → code
```

三點圓弧運動（起點=當前位置，終點 1=pose1，終點 2=pose2）。

參數	說明
<code>pose1</code>	圓弧第二點 <code>[x, y, z, roll, pitch, yaw]</code>
<code>pose2</code>	圓弧第三點 <code>[x, y, z, roll, pitch, yaw]</code>
<code>percent</code>	弧長佔整圓周的百分比（0~100）
<code>is_tool_coord</code>	工具座標系（需 firmware >= 1.11.100）
<code>is_axis_angle</code>	使用軸角表示（需 firmware >= 1.11.100）

```
move_gohome(speed=None, mvacc=None, mvtime=None, is_radian=None, wait=False, timeout=None, **kwargs) → code
```

回到零點（Home 位置）。⚠️ 不帶限位偵測。

參數	說明
<code>speed</code>	速度, 預設 50°/s
<code>mvacc</code>	加速度, 預設 5000°/s²

```
move_arc_lines(paths, is_radian=None, times=1, first_pause_time=0.1,
repeat_pause_time=0, automatic_calibration=True, speed=None, mvacc=None, mvtime=None,
wait=False)
```

連續直線插補運動（支援弧線過渡）。

參數	說明
<code>paths</code>	路徑點 list, 每點格式 <code>[x, y, z, roll, pitch, yaw]</code> 或 <code>[x, y, z, roll, pitch, yaw, radius]</code>
<code>times</code>	重複次數, 0=無限迴圈
<code>first_pause_time</code>	初始等待時間 (s) , 讓指令快取以利連續運動規劃 (預設 0.1s)
<code>repeat_pause_time</code>	每次重複間隔 (s)
<code>automatic_calibration</code>	自動校準 (預設 True)

```
set_position_aa(axis_angle_pose, speed=None, mvacc=None, mvtime=None, is_radian=None,
is_tool_coord=False, relative=False, wait=False, timeout=None, radius=None, **kwargs)
```

→ [code](#)

以**軸角 (axis-angle) **表示法設定目標位姿。

參數	說明
<code>axis_angle_pose</code>	<code>[x(mm), y(mm), z(mm), rx, ry, rz]</code>
<code>is_tool_coord</code>	若為 True, <code>relative</code> 參數無效
<code>radius</code>	需 firmware >= 1.11.100; MoveLineAA (None/< 0) 或 MoveArcLineAA (>= 0)
<code>motion_type</code> (kwargs)	0=線性規劃; 1=優先線性, 不可則關節規劃; 2=直接關節規劃 (需 firmware >= 1.11.100)

```
get_position_aa(is_radian=None) → (code, [x, y, z, rx, ry, rz])
```

取得以軸角表示的目前位姿。

`get_pose_offset(pose1, pose2, orient_type_in=0, orient_type_out=0, is_radian=None) → (code, pose)`

計算兩個位姿之間的偏移量。

參數	說明
<code>pose1/pose2</code>	<code>[x, y, z, roll/rx, pitch/ry, yaw/rz]</code>
<code>orient_type_in</code>	輸入姿態表示: 0=RPY, 1=軸角
<code>orient_type_out</code>	輸出姿態表示: 0=RPY, 1=軸角

`set_servo_attach(servo_id=None) → code`

鎖定 (Attach) 伺服馬達。

參數	說明
<code>servo_id</code>	1~軸數 (鎖定單一關節) 或 8 (鎖定全部)

`set_servo_detach(servo_id=None) → code`

解鎖 (Detach) 伺服馬達。⚠ 解鎖前確保做好保護措施, 避免意外掉臂造成傷害。

參數	說明
<code>servo_id</code>	1~軸數 (解鎖單一關節) 或 8 (解鎖全部)

`emergency_stop()`

緊急停止: 依序執行 `set_state(4)` → `motion_enable(True)` → `set_state(0)`。

⚠ 不會自動清除錯誤, 若有錯誤需依錯誤碼另行處理。

`reset(speed=None, mvacc=None, mvtime=None, is_radian=None, wait=False, timeout=None)`

重置機械臂: 自動清除警告與錯誤, 並在需要時啟用運動與設定狀態。

⚠ 不帶限位偵測, 不修改 `last_used_*` 快取值。

9. 速度控制方法

`vc_set_joint_velocity(speeds, is_radian=None, is_sync=True, duration=-1, **kwargs) → code`

關節速度控制。⚠ 需先呼叫 `set_mode(4)` (需 firmware $\geq 1.6.9$) 。

參數	說明
<code>speeds</code>	<code>[spd_J1, ..., spd_J7]</code> (°/s 或 rad/s)
<code>is_sync</code>	各關節是否同步加減速，預設 True
<code>duration</code>	速度指令持續時間 (s)：> 0=N 秒後自動停止；0=永久生效；< 0=相容舊協議（等同 0）。需 firmware >= 1.8.0
<code>vc_set_cartesian_velocity(speeds, is_radian=None, is_tool_coord=False, duration=-1, **kwargs) → code</code>	

笛卡爾速度控制。⚠️ 需先呼叫 `set_mode(5)`（需 firmware >= 1.6.9）。

參數	說明
<code>speeds</code>	<code>[spd_x(mm/s), spd_y, spd_z, spd_rx, spd_ry, spd_rz]</code>
<code>is_tool_coord</code>	是否使用工具座標系
<code>duration</code>	同 <code>vc_set_joint_velocity</code> ，需 firmware >= 1.8.0

10. 參數設定方法

`set_state(state=0) → code`

設定機械臂狀態。

<code>state</code> 值	說明
0	運動狀態（正常工作）
3	暫停
4	停止
6	減速停止

`set_mode(mode=0, detection_param=0) → code`

設定控制模式（參見模式表格）。

參數	說明
<code>mode</code>	見「狀態與模式屬性」中的 mode 值說明
<code>detection_param</code>	示教偵測：0=開啟動作偵測（預設）；1=關閉。需 firmware >= 1.10.1，且 mode=2 時才有效

`motion_enable(enable=True, servo_id=None) → code`

啟用或停用馬達。

參數	說明
----	----

<code>enable</code>	True=啟用, False=停用
---------------------	-------------------

<code>servo_id</code>	1~軸數 (單一馬達) 或 None/8 (全部)
-----------------------	---------------------------

`set_tcp_offset(offset, is_radian=None, wait=True, **kwargs) → code`

設定末端工具座標偏移。若要清除, 傳入 `[0, 0, 0, 0, 0, 0]`。

參數	說明
----	----

<code>offset</code>	<code>[x(mm), y(mm), z(mm), roll, pitch, yaw]</code>
---------------------	--

<code>wait</code>	等待機械臂停止後才設定
-------------------	-------------

未呼叫 `save_conf()` 則重開機後失效。

`set_tcp_jerk(jerk) → code`

設定笛卡爾空間的平移 jerk (mm/s^3) 。

`set_tcp_maxacc(acc) → code`

設定笛卡爾空間最大加速度 (mm/s^2) 。

`set_joint_jerk(jerk, is_radian=None) → code`

設定關節空間 jerk ($^\circ/\text{s}^3$ 或 rad/s^3) 。

`set_joint_maxacc(acc, is_radian=None) → code`

設定關節空間最大加速度 ($^\circ/\text{s}^2$ 或 rad/s^2) 。

`set_tcp_load(weight, center_of_gravity, wait=False, **kwargs) → code`

設定末端負載。

參數	說明
----	----

<code>weight</code>	負載重量 (kg)
---------------------	-----------

<code>center_of_gravity</code>	重心位置 <code>[x(mm), y(mm), z(mm)]</code>
--------------------------------	---

`set_collision_sensitivity(value, wait=True) → code`

設定碰撞靈敏度（0~5，5 最靈敏）。

`set_teach_sensitivity(value, wait=True) → code`

設定拖動示教靈敏度（1~5）。

`set_gravity_direction(direction, wait=True) → code`

設定重力方向向量（用於力矩補償與碰撞偵測）。

參數	說明
----	----

<code>direction</code>	<code>[x, y, z]</code> ，例如落地安裝 → <code>[0, 0, -1]</code>
------------------------	--

`set_mount_direction(base_tilt_deg, rotation_deg, is_radian=None) → code`

設定機械臂安裝方式（傾斜角、旋轉角）。

`set_world_offset(offset, is_radian=None, wait=True) → code`

設定基座座標偏移（需 firmware >= 1.2.11）。

參數	說明
----	----

<code>offset</code>	<code>[x, y, z, roll, pitch, yaw]</code>
---------------------	--

`set_timeout(timeout)`

設定指令回應逾時時間（秒）。

`clean_conf() → code`

清除目前設定，恢復系統預設值。

`save_conf() → code`

儲存目前設定（重開機後仍有效）。

`set_pause_time(sltime, wait=False) → code`

讓機械臂暫停指定秒數。

參數	說明
----	----

<code>slttime</code>	暫停時間（秒）
----------------------	---------

`set_simulation_robot(on_off) → code`

設定是否為模擬機器人。

`system_control(value=1) → code`

控制控制箱系統。 `value=1`：關機； `value=2`：重開機。

`set_counter_reset() → code`

重設計數器為 0。

`set_counter_increase(val=1) → code`

計數器遞增（目前只支援 +1）。

`set_only_check_type(only_check_type=0)`

設定運動過程的偵測類型（全域配置，影響本 SDK 實例的所有運動介面）。需 firmware >= 1.11.100。

11. 狀態查詢方法

`get_state() → (code, state)`

取得機械臂狀態（1=運動中、2=睡眠、3=暫停、4=停止中）。

`get_is_moving() → bool`

快速查詢是否正在運動。

`get_cmdnum() → (code, cmd_num)`

取得控制器指令快取數量。

`get_err_warn_code(show=False, lang='en') → (code, [error_code, warn_code])`

取得控制器錯誤碼與警告碼。

參數

說明

`show`

是否印出詳細說明

`lang`

顯示語言： `'en'` 或 `'cn'`

`clean_error() → code`

清除錯誤。清除後需手動呼叫 `motion_enable(True)` 及 `set_state(0)`。

`clean_warn() → code`

清除警告。

`get_version() → (code, version)`

取得韌體版本字串。

`get_robot_sn() → (code, sn)`

取得機械臂序號。

`check_verification() → (code, status)`

驗證授權狀態 (0=已驗證) 。

`get_joint_states(is_radian=None, num=3) → (code, [position, velocity, effort])`

取得關節狀態 (需 firmware >= 1.9.0) 。

參數	說明
----	----

<code>num</code>	回傳資料筆數: 1=position; 2=position+velocity; 3=position+velocity+effort
------------------	---

12. 運動學工具方法

`get_inverse_kinematics(pose, input_is_radian=None, return_is_radian=None, limited=True, ref_angles=None) → (code, angles)`

逆向運動學 (IK) : 笛卡爾位姿 → 關節角度。

參數	說明
----	----

<code>pose</code>	<code>[x, y, z, roll, pitch, yaw]</code>
-------------------	--

<code>limited</code>	結果是否限制在 $\pm 180^\circ$ 以內 (需 firmware >= 2.7.103)
----------------------	--

<code>ref_angles</code>	參考關節角度 (需 firmware >= 2.7.103)
-------------------------	--------------------------------

`get_forward_kinematics(angles, input_is_radian=None, return_is_radian=None) → (code, pose)`

正向運動學 (FK) : 關節角度 → 笛卡爾位姿。

參數	說明
----	----

<code>angles</code>	<code>[J1, J2, ..., Jn]</code>
---------------------	--------------------------------

`is_tcp_limit(pose, is_radian=None) → (code, limit)`

檢查笛卡爾位姿是否超出限位 (True=超限, False=未超限, None=失敗) 。

`is_joint_limit(joint, is_radian=None) → (code, limit)`

檢查關節角度是否超出限位（True=超限，False=未超限，None=失敗）。

`get_joints_torque() → (code, joints_torque)`

取得各關節力矩（即時值）。

`set_xarm7_ik_redundancy(jnt_ref, punish_coeff) → code`

設定 xArm7 冗餘自由度 IK 解參數（需 firmware >= 2.7.107）。

參數

說明

`jnt_ref`

參考關節角度，解將盡量靠近此值

`punish_coeff`

懲罰係數

`get_xarm7_ik_redundancy() → (code, params)`

取得 xArm7 冗餘 IK 解參數（需 firmware >= 2.7.107）。

13. 末端夾爪 — xArm Gripper

`set_gripper_enable(enable, **kwargs) → code`

啟用或停用夾爪。`True`=啟用。

`set_gripper_mode(mode, **kwargs) → code`

設定夾爪模式。`mode=0`：位置控制模式。

`get_gripper_position(**kwargs) → (code, pos)`

取得夾爪位置（pulse 單位）。

`get_gripper_g2_position(**kwargs) → (code, pos)`

取得 xArm Gripper G2 位置（mm）。

`set_gripper_position(pos, wait=False, speed=None, auto_enable=False, timeout=None, **kwargs) → code`

設定夾爪位置。

參數	說明
<code>pos</code>	目標位置 (pulse)
<code>speed</code>	速度 (r/min)
<code>auto_enable</code>	自動啟用 (預設 False)
<code>timeout</code>	等待逾時 (s) , 預設 10s

`set_gripper_g2_position(pos, speed=100, force=50, wait=False, timeout=None, **kwargs)`

→ `code`

設定 xArm Gripper G2 位置。

參數	說明
<code>pos</code>	0~84 mm
<code>speed</code>	15~225 mm/s, 預設 100
<code>force</code>	1~100, 預設 50

`set_gripper_speed(speed, **kwargs) → code`

設定夾爪速度 (r/min) 。

`get_gripper_status() → (code, status)`

取得夾爪狀態 (需夾爪 firmware >= 3.4.3) 。

<code>status & 0x03</code>	說明
0	停止
1	運動中
2	已夾持

`get_gripper_err_code(**kwargs) → (code, err_code)`

取得夾爪錯誤碼。

`clean_gripper_error(**kwargs) → code`

清除夾爪錯誤。

14. 末端夾爪 — Vacuum Gripper

`get_vacuum_gripper(hardware_version=1) → (code, state)`

取得真空吸盤狀態。

<code>state</code>	說明
-1	吸盤已關閉
0	吸盤開啟但未吸取物件
1	已吸取物件

<code>hardware_version</code>	說明
1	插件式連接（預設）
2	接觸式連接

`set_vacuum_gripper(on, wait=False, timeout=3, delay_sec=None, sync=True, hardware_version=1) → code`

控制真空吸盤開/關。

參數	說明
<code>on</code>	True=開（等同 <code>tgpio[0]=1, tgpio[1]=0</code> ）； False=關
<code>wait</code>	等待吸取成功
<code>delay_sec</code>	延遲生效時間（s）， None=立即生效
<code>sync</code>	True=在運動佇列中執行； False=立即執行（需 <code>firmware >= 2.4.101</code> ，且 <code>delay_sec <= 0</code> ）

15. 末端夾爪 — Robotiq Gripper

`robotiq_reset(**kwargs) → (code, robotiq_response)`

重置 Robotiq 夾爪（清除先前的啟用狀態）。

`robotiq_set_activate(wait=True, timeout=3, **kwargs) → (code, robotiq_response)`

啟動 Robotiq 夾爪（若尚未啟動）。

```
robotiq_set_position(pos, speed=0xFF, force=0xFF, wait=True, timeout=5, **kwargs) → (code, robotiq_response)
```

設定 Robotiq 夾爪位置。

參數	說明
pos	0 (全開) ~ 255 (全關)
speed	0~255
force	0~255

```
robotiq_open(speed=0xFF, force=0xFF, wait=True, timeout=5, **kwargs) → (code, robotiq_response)
```

開啟 Robotiq 夾爪 (pos=0) 。

```
robotiq_close(speed=0xFF, force=0xFF, wait=True, timeout=5, **kwargs) → (code, robotiq_response)
```

關閉 Robotiq 夾爪 (pos=255) 。

```
robotiq_get_status(number_of_registers=3, **kwargs) → (code, robotiq_response)
```

讀取 Robotiq 狀態暫存器。

number_of_registers	說明
1	只讀 0x07D0 (GRIPPER STATUS)
2	讀 0x07D0、0x07D1 (+ FAULT/POSITION_REQUEST_ECHO)
3	讀 0x07D0~0x07D2 (+ POSITION/CURRENT)

robotiq_status (屬性)

最後一次取得的 Robotiq 狀態 dict:

```
python
```

```

{
    'gOBJ': 0, # 物件偵測狀態
    'gSTA': 0, # 夾爪狀態與運動
    'gGTO': 0, # 動作狀態 (go-to bit)
    'gACT': 0, # 啟用狀態
    'kFLT': 0, # 位置請求 echo
    'gFLT': 0, # 故障狀態
    'gPR': 0, # 請求位置 echo
    'gPO': 0, # 實際位置 (encoder)
    'gCU': 0, # 即時電流
}

```

-1 表示從未更新。

16. 末端夾爪 — BIO Gripper

`set_bio_gripper_enable(enable=True, wait=True, timeout=3, **kwargs) → code`

啟用 BIO 夾爪（若尚未啟用）。

`set_bio_gripper_speed(speed, **kwargs) → code`

設定 BIO 夾爪速度。

`set_bio_gripper_control_mode(mode, **kwargs) → code`

設定 BIO 夾爪控制模式（僅新版 BIO Gripper 支援）。`mode=0`：開合模式；`mode=1`：位置迴路模式。

`set_bio_gripper_force(force, **kwargs) → code`

設定 BIO 夾爪夾力（10~100，僅新版支援）。

`get_bio_gripper_g2_position(**kwargs) → (code, pos)`

取得 BIO Gripper G2 位置（mm）。

`set_bio_gripper_g2_position(pos, speed=2000, force=100, wait=True, timeout=5, **kwargs) → (code, response)`

設定 BIO Gripper G2 位置。

參數	說明
<code>pos</code>	71~150 mm
<code>speed</code>	500~4500 pulse/s, 預設 2000
<code>force</code>	1~100, 預設 100

`open_bio_gripper(speed=0, wait=True, timeout=5, **kwargs) → code`

開啟 BIO 夾爪 (`speed=0` 表示不設定速度)。

`close_bio_gripper(speed=0, wait=True, timeout=5, **kwargs) → code`

關閉 BIO 夾爪。

`get_bio_gripper_status() → (code, status)`

取得 BIO 夾爪狀態。

<code>status & 0x03</code>	說明
0	停止
1	運動中
2	已夾持
3	錯誤

<code>(status >> 2) & 0x03</code>	說明
0	未啟用
1	啟用中
2	已啟用

`get_bio_gripper_error() → (code, error_code)`

取得 BIO 夾爪錯誤碼。

`clean_bio_gripper_error() → code`

清除 BIO 夾爪錯誤。

17. 末端夾爪 — 其他夾爪

DH-PGC-140-50 夾爪

`set_dhpgc_gripper_activate(wait=True, timeout=3, **kwargs) → code`

啟動 DH-PGC-140-50 夾爪。

`set_dhpgc_gripper_position(pos, speed=50, force=50, wait=True, timeout=5, **kwargs) → code`

參數	說明
pos	0~1000
speed	1~100
force	20~100

INSPIRE-ROBOTS RH56DFX 手指

set_rh56_finger_position(finger_id, pos, speed=500, force=500, wait=False, timeout=None, **kwargs) → code

參數	說明
finger_id	1~6（手指編號）
pos	0~1000
speed	0~1000，預設 500
force	0~1000，預設 500

18. Tool GPIO (tgpio)

get_tgpio_digital(ionum=None) → (code, value 或 list)

讀取工具端數位 GPIO 輸入。ionum=0/1 讀取單一；None=同時讀取 0 和 1。

get_tgpio_output_digital(ionum=None) → (code, value 或 list)

讀取工具端數位 GPIO 輸出值。ionum=0/1/2/3/4/None。

get_tool_digital_input(ionum=None) → (code, value 或 list)

讀取工具端數位輸入（與 get_tgpio_digital 差異：不傳 ionum 時取得 TI2 值）。ionum=0~4。

set_tgpio_digital(ionum, value, delay_sec=None, sync=True) → code

設定工具端數位 GPIO 輸出。

參數	說明
<code>ionum</code>	0 或 1
<code>value</code>	輸出值
<code>delay_sec</code>	延遲生效秒數，None=立即
<code>sync</code>	True=在運動佇列執行；False=立即執行（需 firmware >= 2.4.101 且 delay_sec <= 0）

`get_tgpio_analog(ionum=None) → (code, value 或 list)`

讀取工具端類比 GPIO。`ionum=0/1/None`。

`set_tgpio_digital_with_xyz(ionum, value, xyz, fault_tolerance_radius) → code`

當機械臂到達指定 XYZ 位置時，設定工具端數位 GPIO。

參數	說明
<code>ionum</code>	0 或 1
<code>xyz</code>	觸發位置 <code>[x, y, z]</code>
<code>fault_tolerance_radius</code>	容忍半徑（mm）

`config_tgpio_reset_when_stop(on_off) → code`

設定機械臂進入停止狀態時，是否自動重置工具端數位輸出。

`get_tgpio_version() → (code, version)`

取得工具端 GPIO 版本（偵錯用）。

`set_tgpio_monitor_params(io_type, frequency) → code`

設定 TGPIO 監控參數（需 firmware >= 2.7.101）。

參數	說明
<code>io_type</code>	0=關閉監控；其他值見文件
<code>frequency</code>	回報頻率

`get_tgpio_monitor_params() → (code, params)`

取得 TGPIO 監控參數（需 firmware >= 2.7.101）。

19. Controller GPIO (cgpio)

`get_cgpio_digital(ionum=None) → (code, value 或 list)`

讀取控制箱數位 GPIO 輸入。`ionum=0~7` (CI0~CI7) 或 `8~15` (DI0~DI7) 或 `None` (全部)。

`get_cgpio_analog(ionum=None) → (code, value 或 list)`

讀取控制箱類比 GPIO。`ionum=0/1/None`。

`set_cgpio_digital(ionum, value, delay_sec=None, sync=True) → code`

設定控制箱數位 GPIO 輸出。`ionum=0~7` (CO0~CO7) 或 `8~15` (DO0~DO7)。

`set_cgpio_analog(ionum, value, sync=True) → code`

設定控制箱類比 GPIO 輸出。`ionum=0 或 1`。

`set_cgpio_digital_input_function(ionum, fun) → code`

設定控制箱數位**輸入**功能模式。

<code>fun</code>	說明
0	通用輸入
1	外部緊急停止
2	保護重置
11	離線任務
12	示教模式
13	縮減模式
14	啟用手臂

`set_cgpio_digital_output_function(ionum, fun) → code`

設定控制箱數位**輸出**功能模式。

fun	說明
0	通用輸出
1	緊急停止
2	運動中
11	有錯誤
12	有警告
13	碰撞中
14	示教中
15	離線任務中
16	縮減模式中
17	已啟用
18	急停已按下

`get_cgpio_state()` → `(code, states)`

取得控制箱 GPIO 完整狀態。

索引	說明
[0]	GPIO 模組狀態 (0=正常, 1=錯誤, 6=通訊失敗)
[1]	GPIO 模組錯誤碼
[2]	數位輸入功能狀態 (bit mask)
[3]	數位輸入配置狀態 (bit mask)
[4]	數位輸出功能狀態 (bit mask)
[5]	數位輸出配置狀態 (bit mask)
[6]	類比輸入 0
[7]	類比輸入 1
[8]	類比輸出 0
[9]	類比輸出 1
[10]	數位輸入功能模式 list
[11]	數位輸出功能模式 list

`set_cgpio_digital_with_xyz(ionum, value, xyz, fault_tolerance_radius) → code`

當機械臂到達指定 XYZ 位置時，設定控制箱數位 GPIO。 (ionum=0~15)。

`set_cgpio_analog_with_xyz(ionum, value, xyz, fault_tolerance_radius) → code`

當機械臂到達指定 XYZ 位置時，設定控制箱類比 GPIO。 (ionum=0~1)。

`config_cgpio_reset_when_stop(on_off) → code`

設定機械臂停止時，是否自動重置控制箱數位輸出。

20. 六軸力矩感測器 (FT Sensor)

以下功能需 firmware >= 1.8.3 且安裝 UFACTORY 六軸力矩感測器 (不支援第三方)。

`set_ft_sensor_zero() → code`

將當前狀態設為力矩感測器零點。

`set_ft_sensor_enable(on_off) → code`

啟用/停用力矩感測器資料採樣。

`set_ft_sensor_mode(mode, **kwargs) → code`

設定力控模式。`mode=0`：非力控模式；`1`：導納控制；`2`：力控模式。

`get_ft_sensor_mode() → (code, mode)`

取得當前力控模式。

`get_ft_sensor_data(is_raw=False) → (code, ft_data)`

取得力矩感測器資料（補償後外力值）。`is_raw=True` 回傳原始值（需 firmware $\geq$ 2.6.109）。

`get_ft_sensor_config() → (code, config)`

取得力矩感測器完整設定（模式、負載、零點偏移、導納參數等）。

`get_ft_sensor_error() → (code, error)`

取得力矩感測器錯誤碼。

`iden_ft_sensor_load_offset() → (code, load_offset)`

用力矩感測器自動識別 TCP 負載與偏移。

回傳：`[mass(kg), x_centroid(mm), y_centroid(mm), z_centroid(mm), Fx_offset, Fy_offset, Fz_offset, Tx_offset, Ty_offset, Tz_offset]`

`set_ft_sensor_load_offset(iden_result_list, association_setting_tcp_load=False, **kwargs) → code`

寫入力矩感測器識別到的負載與偏移參數。

參數	說明
<code>association_setting_tcp_load</code>	True=同時更新 TCP 負載設定

`set_ft_sensor_admittance_parameters(coord=None, c_axis=None, M=None, K=None, B=None, **kwargs) → code`

設定導納控制參數（用於柔順控制）。

參數	說明
<code>coord</code>	任務座標系：0=基座座標，1=工具座標
<code>c_axis</code>	6D 向量（0/1），1 表示該軸進行導納控制
<code>M</code>	6D 質量向量（kg）
<code>K</code>	6D 剛度係數
<code>B</code>	6D 阻尼係數（控制器內部設為 $2*\sqrt{M*K}$ ）

`set_ft_sensor_force_parameters(coord=None, c_axis=None, f_ref=None, limits=None, kp=None, ki=None, kd=None, xe_limit=None, **kwargs) → code`

設定力控模式參數。

參數	說明
<code>f_ref</code>	6D 向量，目標力/力矩（機械臂調整位置以達到此力）
<code>limits</code>	6D 向量，柔順軸的最大 TCP 速度
<code>kp/ki/kd</code>	6D PID 增益
<code>xe_limit</code>	6D 最大允許 TCP 速度（mm/s）

`set_ft_collision_detection(on_off) → code`

啟用力矩感測器碰撞偵測（需 firmware >= 2.6.103）。

`set_ft_collision_rebound(on_off) → code`

啟用力矩感測器碰撞反彈（需 firmware >= 2.6.103）。

`set_ft_collision_threshold(thresholds) → code`

設定碰撞偵測閾值。`thresholds=[x(N), y(N), z(N), Rx(Nm), Ry(Nm), Rz(Nm)]`，x 範圍：5~200N。

`set_ft_collision_reb_distance(distances, is_radian=None) → code`

設定碰撞反彈距離。`distances=[x(mm), y(mm), z(mm), Rx, Ry, Rz]`。

`set_ft_admittance_ctrl_threshold(thresholds) → code`

設定導納控制各方向反應閾值（需 firmware >= 2.6.110）。

`get_ft_collision_detection() → (code, on_off)`

查詢碰撞偵測是否啟用。

`get_ft_collision_rebound() → (code, on_off)`

查詢碰撞反彈是否啟用。

`get_ft_collision_threshold() → (code, thresholds)`

取得碰撞偵測閾值。

`get_ft_collision_reb_distance(is_radian=None) → (code, distances)`

取得碰撞反彈距離。

`get_ft_admittance_ctrl_threshold() → (code, thresholds)`

取得導納控制閾值（需 firmware >= 2.6.110）。

`iden_tcp_load(estimated_mass=0) → (code, load)`

用電流識別 TCP 負載（需 firmware >= 1.8.0）。

⚠ Lite6 機型**必須**指定 `estimated_mass`。

回傳： `[mass, x_centroid, y_centroid, z_centroid]`

21. Linear Motor（線性馬達）

┆ 以下功能需 firmware >= 1.8.0。

`get_linear_motor_registers(**kwargs) → (code, status_dict)`

取得線性馬達完整狀態（position、status、error、is_enabled、on_zero、sci、sco）。

`get_linear_motor_pos() → (code, position)`

取得線性馬達位置。

`get_linear_motor_status() → (code, status)`

取得線性馬達狀態（0=完成，1=運動中，2=已停止）。

`get_linear_motor_error() → (code, error)`

取得線性馬達錯誤碼。

`get_linear_motor_is_enabled() → (code, status)`

查詢是否已啟用（0=未啟用，1=啟用）。

`get_linear_motor_on_zero() → (code, status)`

查詢是否在零點（0=否，1=是）。

`get_linear_motor_sci() → (code, sci1)` / `get_linear_motor_sco() → (code, sco)`

取得 SCI/SCO 狀態。

`clean_linear_motor_error() → code`

清除線性馬達錯誤。

`set_linear_motor_enable(enable) → code`

啟用/停用線性馬達。

`set_linear_motor_speed(speed) → code`

設定線性馬達速度 (1~1000 mm/s) 。

`set_linear_motor_back_origin(wait=True, **kwargs) → code`

使線性馬達回零點 (首次上電必須執行) 。

`set_linear_motor_pos(pos, speed=None, wait=True, timeout=100, **kwargs) → code`

設定線性馬達目標位置。

參數	說明
----	----

<code>pos</code>	0~700/1000/1500 mm (依 SN 型號而定; AL1300=0~700mm, AL1301=另有範圍)
------------------	---

`set_linear_motor_stop() → code`

停止線性馬達。

22. Callback 回呼管理

`register_report_callback(callback, report_cartesian=True, report_joints=True, report_state=True, report_error_code=True, report_warn_code=True, report_mtable=True, report_mtbrake=True, report_cmd_num=True) → bool`

註冊通用回報 callback (需 enable_report=True) 。

callback 資料格式：

```
python
```

```
{
    'cartesian': [x, y, z, roll, pitch, yaw], # 若 report_cartesian=True
    'joints': [J1, ..., J7],                # 若 report_joints=True
    'error_code': 0,
    'warn_code': 0,
    'state': state,
    'mtbrake': [...],
    'mtable': [...],
    'cmdnum': 0,
}
```

register_report_location_callback(callback, report_cartesian=True, report_joints=True) → bool

只回報位置的 callback。

python

```
{'cartesian': [x, y, z, roll, pitch, yaw], 'joints': [J1, ..., J7]}
```

register_connect_changed_callback(callback) → bool

連線狀態變更時觸發。({'connected': bool, 'reported': bool})

register_state_changed_callback(callback) → bool

機械臂狀態變更時觸發。({'state': int})

register_mode_changed_callback(callback) → bool

控制模式變更時觸發（需 socket + enable_report）。({'mode': int})

register_mtable_mtbrake_changed_callback(callback) → bool

馬達啟用/剎車狀態變更時觸發。({'mtable': [...], 'mtbrake': [...]})

register_error_warn_changed_callback(callback) → bool

錯誤或警告碼變更時觸發。({'error_code': int, 'warn_code': int})

register_cmdnum_changed_callback(callback) → bool

指令佇列數量變更時觸發。({'cmdnum': int})

register_temperature_changed_callback(callback) → bool

溫度變更時觸發。({'temperatures': [T1, ..., T7]})

`register_count_changed_callback(callback) → bool`

計數器變更時觸發。 `{'count': int}`

`register_iden_progress_changed_callback(callback) → bool`

識別進度變更時觸發。 `{'progress': int}`

`register_feedback_callback(callback=None) → bool`

Feedback callback（需 firmware $\geq 2.1.0$ ）。資料為 bytes。

Release 系列（每個 register 對應一個 release）

方法	說明
<code>release_report_callback(callback)</code>	解除通用回報 callback
<code>release_report_location_callback(callback)</code>	解除位置回報 callback
<code>release_connect_changed_callback(callback)</code>	解除連線狀態 callback
<code>release_state_changed_callback(callback)</code>	解除狀態 callback
<code>release_mode_changed_callback(callback)</code>	解除模式 callback
<code>release_mtable_mtbrake_changed_callback(callback)</code>	解除馬達狀態 callback
<code>release_error_warn_changed_callback(callback)</code>	解除錯誤警告 callback
<code>release_cmdnum_changed_callback(callback)</code>	解除佇列數量 callback
<code>release_temperature_changed_callback(callback)</code>	解除溫度 callback
<code>release_count_changed_callback(callback)</code>	解除計數器 callback
<code>release_iden_progress_changed_callback(callback)</code>	解除識別進度 callback
<code>release_feedback_callback(callback)</code>	解除 feedback callback

23. 軌跡錄製與回放

┆ 需 firmware $\geq 1.2.0$ ，搭配 xArmStudio $\geq 1.2.0$ 。

`get_trajectories() → (code, trajectories)`

取得已存軌跡清單。每筆： `{'name': str, 'duration': float(秒)}`。

`start_record_trajectory() → code`

開始錄製軌跡（需先呼叫 `set_mode(2)` 和 `set_state(0)` 進入示教模式）。

`stop_record_trajectory(filename=None, **kwargs) → code`

停止錄製。

參數	說明
----	----

<code>filename</code>	儲存名稱（僅英文或數字，最長 50 字元）；None=僅停止不儲存
-----------------------	-----------------------------------

`get_record_seconds() → (code, seconds)`

取得錄製秒數（需 firmware >= 2.4.0，錄製中或錄製完成但尚未儲存時有效）。

`save_record_trajectory(filename, wait=True, timeout=5, **kwargs) → code`

儲存當前錄製的軌跡至控制箱。⚠ 重複呼叫會以空軌跡覆蓋。

`load_trajectory(filename, wait=True, timeout=None, **kwargs) → code`

載入指定軌跡至記憶體。

`playback_trajectory(times=1, filename=None, wait=True, double_speed=1, **kwargs) → code`

播放軌跡。

參數	說明
----	----

<code>times</code>	重複次數（僅在當前位置=軌跡結束位置時有效，否則只播放一次）
--------------------	--------------------------------

<code>filename</code>	若為 None，需先呼叫 <code>load_trajectory</code>
-----------------------	---

<code>double_speed</code>	倍速播放：1/2/4（需 firmware > 1.2.11）
---------------------------	---------------------------------

`get_trajectory_rw_status() → (code, status)`

取得軌跡讀寫狀態（0=無、1=載入中、2=載入成功、3=載入失敗、4=儲存中、5=儲存成功、6=儲存失敗）。

`get_record_seconds() → (code, seconds)`

取得實際錄製時長（需 firmware >= 2.4.0）。

`delete_trajectory(name) → code`

刪除指定軌跡。

`get_traj_speeding(rate) → (code, speed_info)`

取得軌跡錄製中超速的關節與速度資訊。`rate=1/2/4`。

24. 校正工具方法

需 firmware $\geq 1.6.9$ 。

`calibrate_tcp_coordinate_offset(four_points, is_radian=None) → (code, [x, y, z])`

四點法校正 TCP 位置偏移。

`four_points`: 4 個示教位置 list, 每點 `[x, y, z, roll, pitch, yaw]`。

`calibrate_tcp_orientation_offset(rpy_be, rpy_bt, input_is_radian=None, return_is_radian=None) → (code, [roll, pitch, yaw])`

額外示教點校正 TCP 姿態偏移。

參數	說明
<code>rpy_be</code>	無 TCP 偏移時的示教點 RPY
<code>rpy_bt</code>	有 TCP 偏移時的示教點 RPY

`calibrate_user_orientation_offset(three_points, mode=0, trust_ind=0, ...) → (code, [roll, pitch, yaw])`

三點法示教使用者座標系姿態偏移。

點位選取規則：第一點為原點，第二點沿目標座標系 X+ 方向，第三點沿 Y+ 方向（允許不完全垂直，計算時自動修正）。

`calibrate_user_coordinate_offset(rpy_ub, pos_b_uorg, is_radian=None) → (code, [x, y, z])`

確定使用者座標系位置偏移（一個額外示教點）。

參數	說明
<code>rpy_ub</code>	<code>calibrate_user_orientation_offset</code> 的結果
<code>pos_b_uorg</code>	示教點在基座座標系的位置（若機械臂無法到達，可手動輸入）

25. RS485 / Modbus RTU

`set_rs485_baudrate(baud, target='robot', **kwargs) → (code, baudrate)`

設定 RS485 鮑率。

參數	說明
----	----

baud	支援： 4800/9600/19200/38400/57600/115200/230400/460800/921600/1000000/1500000/2000000/2500000
------	--

target	'robot' (機器人 RS485) 或 'control_box' (控制箱 RS485)
--------	---

`get_rs485_baudrate(target='robot', **kwargs) → (code, baudrate)`

取得 RS485 鮑率。

`set_rs485_timeout(timeout, target='robot', protocol='modbus_rtu', **kwargs) → code`

設定 RS485 逾時 (毫秒)。`protocol='modbus_rtu'` 或 `'transparent'`。

`get_rs485_timeout(target='robot', protocol='modbus_rtu', **kwargs) → (code, timeout)`

取得 RS485 逾時 (毫秒)。

`set_rs485_data(datas, min_res_len=0, target='robot', protocol='modbus_rtu', use_503_port=False, **kwargs) → (code, modbus_response)`

發送 Modbus 資料至 RS485。

參數	說明
----	----

datas	資料 list
-------	---------

min_res_len	最小回應長度 (0=不檢查)
-------------	----------------

use_503_port	使用 port 503 通訊 (透明傳輸用, 需 firmware >= 1.11.0)
--------------	--

`set_modbusrtu_params(slave_id, baudrate, stopbits=1, parity=0) → code`

設定 Modbus RTU 參數。

參數	說明
----	----

slave_id	1~247
----------	-------

stopbits	1 或 2
----------	-------

parity	同位元
--------	-----

`get_modbusrtu_params() → (code, [slave_id, baudrate, stopbits, parity])`

取得 Modbus RTU 參數。

`getset_tgpio_modbus_data(datas, min_res_len=0, ...) → (code, response)`

發送 Modbus 資料至 RS485 (舊版介面, 建議改用 `set_rs485_data`) 。

`set_baud_checkset_enable(enable) → code`

啟用/停用末端 IO 板自動鮑率校驗 (影響 gripper/bio/robotiq/linear_motor) 。

`set_checkset_default_baud(type_, baud) → code`

設定校驗鮑率值。 `type_=1` (xArm Gripper) / `2` (BIO) / `3` (Robotiq) / `4` (Linear Motor) ; `baud <= 0` 停用校驗。

`get_checkset_default_baud(type_) → (code, baud)`

取得校驗鮑率值。

26. Modbus TCP (標準 Modbus)

`read_coil_bits(addr, quantity) → (code, bits)`

讀取線圈 (Function Code 0x01) 。

`read_input_bits(addr, quantity) → (code, bits)`

讀取離散輸入 (Function Code 0x02) 。

`read_holding_registers(addr, quantity, is_signed=False) → (code, regs)`

讀取保持暫存器 (Function Code 0x03) 。

`read_input_registers(addr, quantity, is_signed=False) → (code, regs)`

讀取輸入暫存器 (Function Code 0x04) 。

`write_single_coil_bit(addr, bit_val) → code`

寫入單一線圈 (Function Code 0x05) 。 `bit_val=0/1`。

`write_single_holding_register(addr, reg_val) → code`

寫入單一保持暫存器 (Function Code 0x06) 。

`write_multiple_coil_bits(addr, bits) → code`

寫入多個線圈 (Function Code 0x0F) 。

`write_multiple_holding_registers(addr, regs) → code`

寫入多個保持暫存器 (Function Code 0x10) 。

`mask_write_holding_register(addr, and_mask, or_mask) → code`

遮罩寫入保持暫存器 (Function Code 0x16) 。

`write_and_read_holding_registers(r_addr, r_quantity, w_addr, w_regs, is_signed=False)
→ (code, regs)`

寫入並讀取保持暫存器 (Function Code 0x17) 。

`send_hex_cmd(datas, **kwargs) → code 或 hex_list`

發送十六進位協議指令。 `(datas)`: 十六進位資料 list。

27. 縮減模式與安全邊界

`get_reduced_mode() → (code, mode)`

取得縮減模式狀態 (0=關閉, 1=開啟) 。

`set_reduced_mode(on) → code`

開啟/關閉縮減模式 (需 firmware >= 1.2.0) 。

`get_reduced_states(is_radian=None) → (code, states)`

取得縮減模式的完整設定狀態 (需 firmware >= 1.2.0) 。

若 firmware > 1.2.11, states 包含: `[reduced_on, [x_max, x_min, y_max, y_min, z_max, z_min], max_tcp_speed, max_joint_speed, joint_ranges, fence_on, collision_rebound_on]`

`set_reduced_max_tcp_speed(speed) → code`

設定縮減模式最大 TCP 速度 (mm/s) 。需重新啟動縮減模式生效。

`set_reduced_max_joint_speed(speed, is_radian=None) → code`

設定縮減模式最大關節速度。需重新啟動縮減模式生效。

`set_reduced_tcp_boundary(boundary) → code`

設定安全邊界。 `boundary=[x_max, x_min, y_max, y_min, z_max, z_min]` (mm) 。

`set_reduced_joint_range(joint_range, is_radian=None) → code`

設定縮減模式下的關節限位 (需 firmware >= 1.2.11) 。

`joint_range=[J1_min, J1_max, ..., J7_min, J7_max]`

`set_fence_mode(on) → code`

開啟/關閉安全邊界模式 (需 firmware >= 1.2.11) 。

`set_collision_rebound(on) → code`

開啟/關閉碰撞反彈（需 firmware >= 1.2.11）。

`set_self_collision_detection(on_off) → code`

開啟/關閉自碰撞偵測。

`set_collision_tool_model(tool_type, *args, **kwargs) → code`

設定末端工具幾何模型（用於自碰撞偵測）。

`tool_type` **工具**

0	無末端工具
1	xArm Gripper
2	xArm Vacuum Gripper
3	xArm BIO Gripper
4	Robotiq-2F-85
5	Robotiq-2F-140
7	Lite Gripper
8	Lite Vacuum Gripper
9	xArm Gripper G2
10	DH-ROBOTICS PGC-140-50
11	INSPIRE-ROBOTS RH56DFX-2L
12	INSPIRE-ROBOTS RH56DFX-2R
13	xArm BIO Gripper G2
21	圓柱體（需指定 <code>radius(mm)</code> , <code>height(mm)</code> , <code>x_offset</code> , <code>y_offset</code> , <code>z_offset</code> ）
22	長方體（需指定 <code>x(mm)</code> , <code>y(mm)</code> , <code>z(mm)</code> , <code>x_offset</code> , <code>y_offset</code> , <code>z_offset</code> ）

`set_report_tau_or_i(tau_or_i=0) → code`

設定回報力矩（0）或電流（1）。

`get_report_tau_or_i() → (code, tau_or_i)`

取得目前回報類型。

28. 進階診斷與錯誤資訊

`get_c31_error_info()` → `(code, err_info)`

取得碰撞錯誤 (C31) 詳細資訊 (需 firmware $\geq$ 2.3.0) 。

`get_c37_error_info(is_radian=None)` → `(code, err_info)`

取得負載錯誤 (C37) 詳細資訊。

`get_c23_error_info(is_radian=None)` → `(code, err_info)`

取得關節角度超限錯誤 (C23) 詳細資訊。

`get_c24_error_info(is_radian=None)` → `(code, err_info)`

取得關節速度超限錯誤 (C24) 詳細資訊。

`get_c60_error_info()` → `(code, err_info)`

取得線速度超限錯誤 (C60) 詳細資訊 (需 firmware $\geq$ 2.3.0, mode 1) 。

`get_c38_error_info(is_radian=None)` → `(code, err_info)`

取得關節硬體角度超限錯誤 (C38) 詳細資訊 (需 firmware $\geq$ 2.4.0) 。

`get_c54_error_info()` → `(code, err_info)`

取得力矩感測器碰撞錯誤 (C54) 詳細資訊 (需 firmware $\geq$ 2.6.103) 。

`get_poe_status()` → `(code, status)`

取得 PoE 狀態 (需 firmware $\geq$ 2.3.0) 。

`get_iden_status()` → `(code, status)`

取得識別狀態 (需 firmware $\geq$ 2.3.0) 。

`get_servo_debug_msg(show=False, lang='en')` → `(code, servo_info_list)`

取得伺服偵錯訊息 (僅供偵錯用) 。

29. DH 參數、回饋、其他進階功能

`get_dh_params()` → `(code, dh_params)`

取得 DH 參數 (需 firmware $\geq$ 2.0.0) 。

`set_dh_params(dh_params, flag=0)` → `code`

設定 DH 參數 (需 firmware $\geq$ 2.0.0) 。 ⚠ 僅供需要外部 DH 參數的使用者，一般使用者不應修改。

`set_feedback_type(feedback_type) → code`

設定回饋類型（需 firmware $\geq$ 2.1.0，僅在 position mode 有效，只影響後續任務）。

`set_cartesian_velo_continuous(on_off) → code`

設定笛卡爾運動速度是否連續（需 firmware $\geq$ 1.9.0）。

`set_allow_approx_motion(on_off) → code`

允許在某些奇異點附近使用近似解以避免超速（需 firmware $\geq$ 1.9.0）。

`get_allow_approx_motion() → code`

取得是否啟用近似解（需 firmware $\geq$ 1.9.0）。

`iden_joint_friction(sn=None) → (code, result)`

識別關節摩擦力（需 firmware $\geq$ 1.9.0）。

`set_linear_spd_limit_factor(factor) → code`

設定線速度限制係數（預設 1.2，需 firmware $\geq$ 2.3.0，mode 1 才有效）。

`get_linear_spd_limit_factor() → (code, factor)`

取得線速度限制係數。

`set_cmd_mat_history_num(num) → code`

設定指令矩陣歷史數量（需 firmware $\geq$ 2.3.0）。

`get_cmd_mat_history_num() → (code, num)`

取得指令矩陣歷史數量。

`set_fdb_mat_history_num(num) → code`

設定回饋矩陣歷史數量。

`get_fdb_mat_history_num() → (code, num)`

取得回饋矩陣歷史數量。

`get_base_board_version(board_id=10) → (code, version)`

取得底板版本。

`get_initial_point() → (code, point)`

從 Studio 取得初始點。`point=[J1, J2, ..., J7]`。

`set_initial_point(point) → code`

設定初始點。

`get_mount_direction() → (code, [tilt_angle, rotate_angle])`

取得安裝方向角度。

`set_external_device_monitor_params(dev_type, frequency) → code`

設定外部裝置監控參數（需 firmware $\geq$ 2.7.100）。啟動後外部裝置的位置/速度/電流資訊將透過 port 30000 回報。

`get_external_device_monitor_params() → (code, params)`

取得外部裝置監控參數。

`run_blockly_app(path, **kwargs)`

執行 xArmStudio 生成的 Blockly 應用程式。

`run_gcode_file(path, **kwargs)`

執行 G-code 檔案。

`run_gcode_app(path, **kwargs) → code`

執行 xArmStudio G-code 專案檔。

30. Lite6 專屬夾爪

■ 需 firmware $\geq$ 1.10.0，僅適用於 Lite6 系列機械臂。

`open_lite6_gripper(sync=True) → code`

開啟 Lite6 夾爪。`sync=True` 表示在運動佇列中執行；`False` 立即執行。

`close_lite6_gripper(sync=True) → code`

關閉 Lite6 夾爪。

`stop_lite6_gripper(sync=True) → code`

停止 Lite6 夾爪。

31. 偵錯與版本查詢

方法	說明
<code>get_gripper_version() → (code, version)</code>	取得夾爪版本（偵錯用）
<code>get_servo_version(servo_id=1) → (code, version)</code>	取得指定伺服版本（servo_id=1~7）
<code>get_harmonic_type(servo_id=1) → (code, type)</code>	取得諧波類型（偵錯用）
<code>get_hd_types() → (code, types)</code>	取得所有諧波類型（偵錯用）
<code>set_rs485_use_503_port(use_503_port=True)</code>	設定是否使用 port 503 進行 RS485 通訊
<code>set_tgpio_modbus_timeout(...)</code>	舊版介面，請改用 <code>set_rs485_timeout</code>
<code>get_tgpio_modbus_timeout(...)</code>	舊版介面，請改用 <code>get_rs485_timeout</code>
<code>set_tgpio_modbus_baudrate(baud)</code>	舊版介面，請改用 <code>set_rs485_baudrate</code>
<code>get_tgpio_modbus_baudrate()</code>	舊版介面，請改用 <code>get_rs485_baudrate</code>

32. 屬性別名 (Alias)

以下別名在 `__getattr__` 中對應到實際方法，可互換使用：

別名	實際方法/屬性
<code>get_ik</code>	<code>get_inverse_kinematics</code>
<code>get_fk</code>	<code>get_forward_kinematics</code>
<code>set_sleep_time</code>	<code>set_pause_time</code>
<code>register_maable_mtbrake_changed_callback</code>	<code>register_mtable_mtbrake_changed_callback</code>
<code>release_maable_mtbrake_changed_callback</code>	<code>release_mtable_mtbrake_changed_callback</code>
<code>position_offset</code>	<code>tcp_offset</code> （屬性）
<code>get_gpio_digital</code>	<code>get_tgpio_digital</code>
<code>set_gpio_digital</code>	<code>set_tgpio_digital</code>

常見使用模式 (針對本專案 sEMG → Lite6 控制)

```
python

from xarm.wrapper import XArmAPI

arm = XArmAPI('192.168.1.x', is_radian=False)
arm.motion_enable(True)
arm.set_mode(0)          # 位置控制模式
arm.set_state(0)         # 運動狀態

# 移動到目標位置 (等待完成)
code = arm.set_position(x=300, y=0, z=200, roll=180, pitch=0, yaw=0, wait=True)

# 控制 Lite6 夾爪 (sEMG 觸發)
arm.open_lite6_gripper(sync=True)    # 對應 ED (伸指) 動作
arm.close_lite6_gripper(sync=True)   # 對應 FDS (屈指) 動作

# 清除錯誤並恢復
arm.clean_error()
arm.motion_enable(True)
arm.set_state(0)

arm.disconnect()
```

本文件由 `xarm_api.py` (Ufactory xArm Python SDK) 原始碼完整解析生成, 2026/07/26。